

طراحی الگوریتم‌ها – الگوریتم‌های درخت پوشای کمینه

Introduction to Algorithm

مهدی جوانمردی

فصل ۲۳ کتاب – درخت پوشای کمینه

• ۲۳.۱: رشد درخت پوشای کمینه

• ۲۳.۲: الگوریتم‌های Kruskal و Prim

23 Minimum Spanning Trees 624

23.1 Growing a minimum spanning tree 625

23.2 The algorithms of Kruskal and Prim 631

تعریف درخت پوشای کمینه یا MST

• درخت پوشا یا Spanning tree

• با در اختیار داشتن یک گراف متصل با n گره، یک درخت پوشا عبارت است از درختی با $n-1$ یال که همه n گره را به هم متصل کند

آن n گره و $n-1$ یال یک زیرگراف متصل را تشکیل می‌دهد

• درخت پوشا یکتا نیست و ممکن است چندین درخت پوشا بتوان تعریف کرد

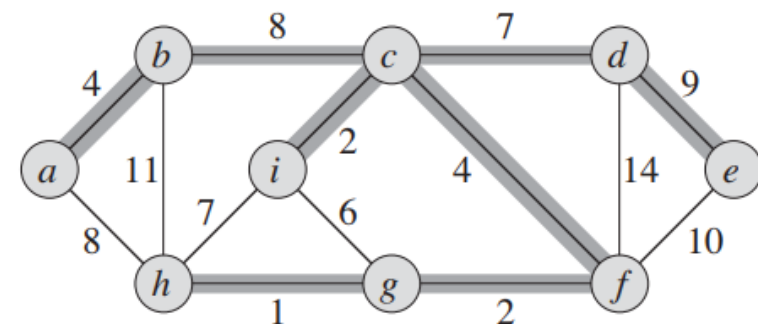
• درخت پوشای کمینه:

$$G = (V, E)$$

edge $(u, v) \in E \longrightarrow w(u, v)$ weight

find an acyclic subset $T \subseteq E$ that connects all of the vertices and whose total weight

$$w(T) = \sum_{(u,v) \in T} w(u, v) \text{ is minimized.}$$



ایجاد یک درخت پوشای کمینه

• روش حریصانه برای ایجاد یک درخت پوشای کمینه برای گراف وزن دار $G = (V, E)$ با $w : E \rightarrow \mathbb{R}$

maintaining the following loop invariant:

Prior to each iteration, A is a subset of some minimum spanning tree.

در هر مرحله یک یال (u, v) را بگونه ای انتخاب میکنیم که $A \cup \{(u, v)\}$ کماکان یک MST باشد

We call such an edge a *safe edge* for A

GENERIC-MST(G, w)

```
1   $A = \emptyset$ 
2  while  $A$  does not form a spanning tree
3      find an edge  $(u, v)$  that is safe for  $A$ 
4       $A = A \cup \{(u, v)\}$ 
5  return  $A$ 
```

ایجاد یک درخت پوشای کمینه

GENERIC-MST(G, w)

```
1   $A = \emptyset$ 
2  while  $A$  does not form a spanning tree
3      find an edge  $(u, v)$  that is safe for  $A$ 
4       $A = A \cup \{(u, v)\}$ 
5  return  $A$ 
```

Initialization: After line 1, the set A trivially satisfies the loop invariant.

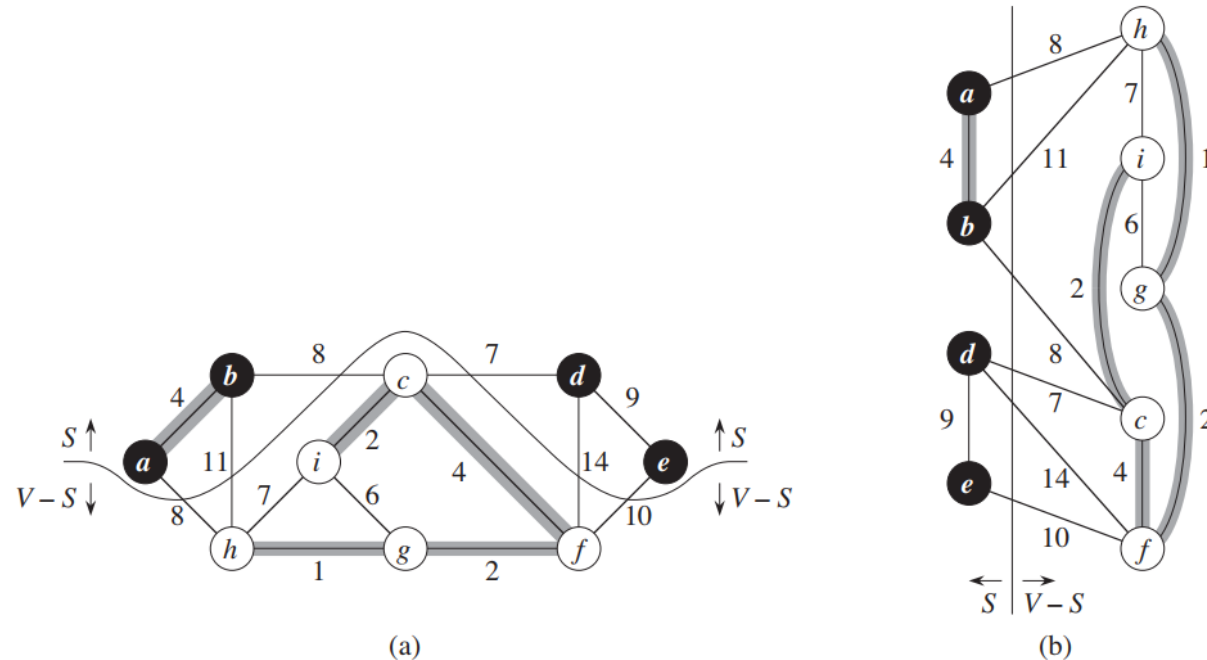
Maintenance: The loop in lines 2–4 maintains the invariant by adding only safe edges.

Termination: All edges added to A are in a minimum spanning tree, and so the set A returned in line 5 must be a minimum spanning tree.

The tricky part is, of course, finding a safe edge in line 3.

تعاریف در فرایند ایجاد درخت پوشای کمینه

- برخی تعاریف مورد نیاز برای ادامه: برش گراف $G = (V, E)$ با نماد $cut(S, V - S)$



an edge $(u, v) \in E$ **crosses** the cut $(S, V - S)$ if one of its endpoints is in S and the other is in $V - S$.

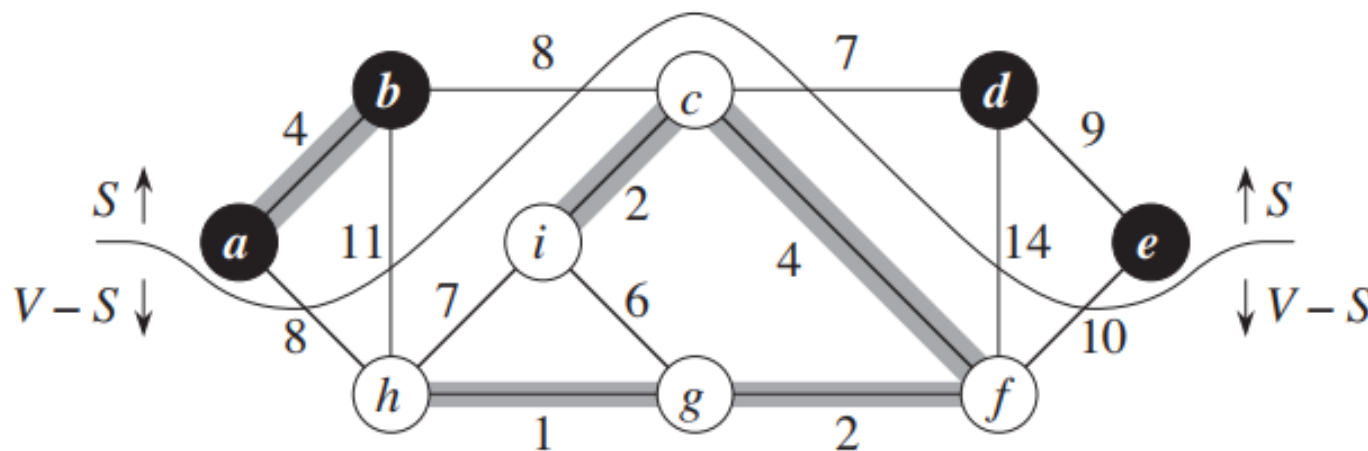
a cut **respects** a set A of edges if no edge in A crosses the cut

An edge is a **light edge** crossing a cut if its weight is the minimum of any edge crossing the cut

قضیه یال ایمن

Theorem 23.1

Let $G = (V, E)$ be a connected, undirected graph with a real-valued weight function w defined on E . Let A be a subset of E that is included in some minimum spanning tree for G , let $(S, V - S)$ be any cut of G that respects A , and let (u, v) be a light edge crossing $(S, V - S)$. Then, edge (u, v) is safe for A .



اثبات قضیه یال ایمن

Proof Let T be a minimum spanning tree that includes A , and assume that T does not contain the light edge (u, v) , since if it does, we are done. We shall construct another minimum spanning tree T' that includes $A \cup \{(u, v)\}$ by using a cut-and-paste technique, thereby showing that (u, v) is a safe edge for A .

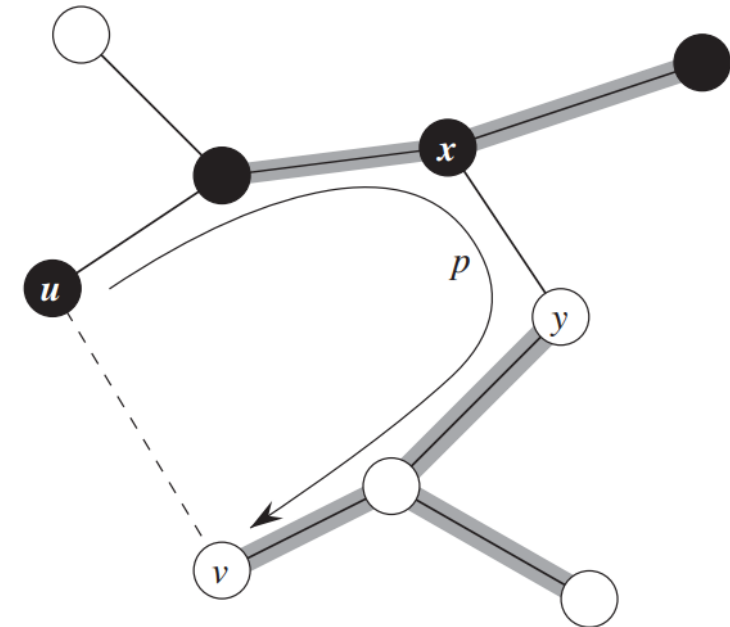
The edge (u, v) forms a cycle with the edges on the simple path p from u to v in T , as Figure 23.3 illustrates. Since u and v are on opposite sides of the cut $(S, V - S)$, at least one edge in T lies on the simple path p and also crosses the cut. Let (x, y) be any such edge. The edge (x, y) is not in A , because the cut respects A . Since (x, y) is on the unique simple path from u to v in T , removing (x, y) breaks T into two components. Adding (u, v) reconnects them to form a new spanning tree $T' = T - \{(x, y)\} \cup \{(u, v)\}$.

We next show that T' is a minimum spanning tree. Since (u, v) is a light edge crossing $(S, V - S)$ and (x, y) also crosses this cut, $w(u, v) \leq w(x, y)$. Therefore,

$$\begin{aligned} w(T') &= w(T) - w(x, y) + w(u, v) \\ &\leq w(T). \end{aligned}$$

But T is a minimum spanning tree, so that $w(T) \leq w(T')$; thus, T' must be a minimum spanning tree also.

It remains to show that (u, v) is actually a safe edge for A . We have $A \subseteq T'$, since $A \subseteq T$ and $(x, y) \notin A$; thus, $A \cup \{(u, v)\} \subseteq T'$. Consequently, since T' is a minimum spanning tree, (u, v) is safe for A . ■



Black vertices are in S , and white vertices are in $V - S$.

The edges in the minimum spanning tree T ———

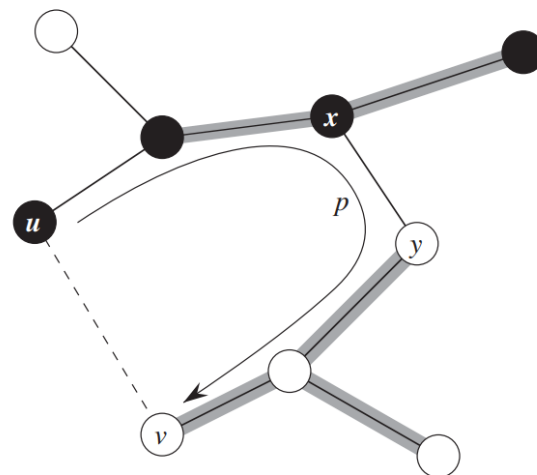
edges in A ———

ایده اصلی الگوریتم های MST

Corollary 23.2

Let $G = (V, E)$ be a connected, undirected graph with a real-valued weight function w defined on E . Let A be a subset of E that is included in some minimum spanning tree for G , and let $C = (V_C, E_C)$ be a connected component (tree) in the forest $G_A = (V, A)$. If (u, v) is a light edge connecting C to some other component in G_A , then (u, v) is safe for A .

Proof The cut $(V_C, V - V_C)$ respects A , and (u, v) is a light edge for this cut. Therefore, (u, v) is safe for A . ■



Black vertices are in S , and white vertices are in $V - S$.

The edges in the minimum spanning tree T ———

edges in A ==

الگوریتم Kruskal برای ایجاد MST

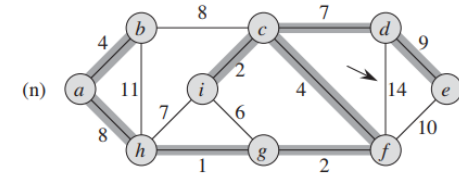
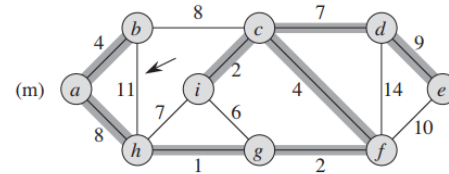
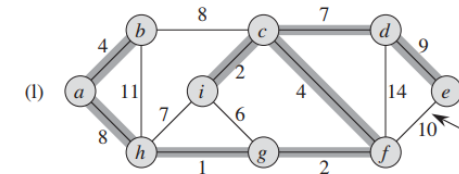
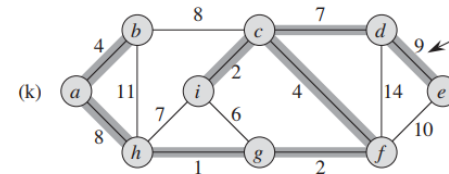
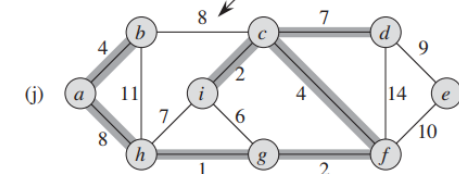
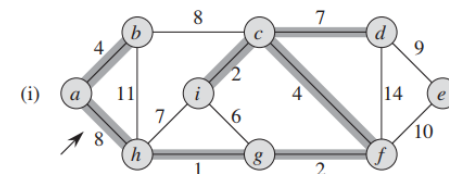
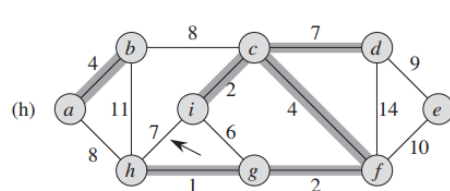
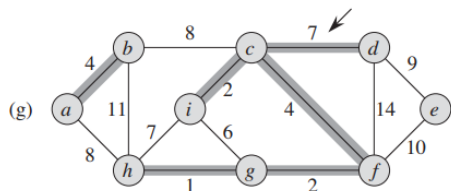
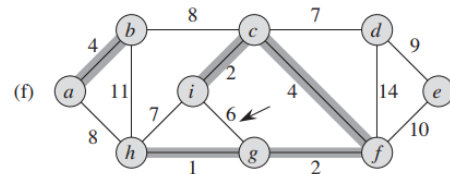
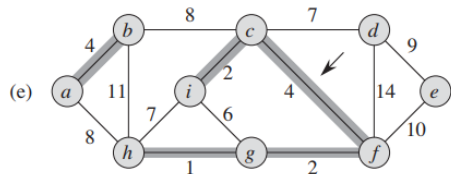
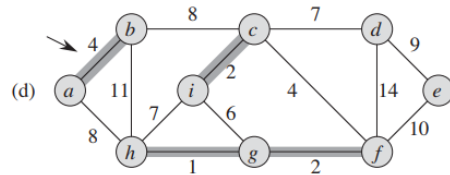
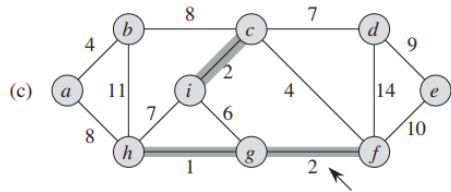
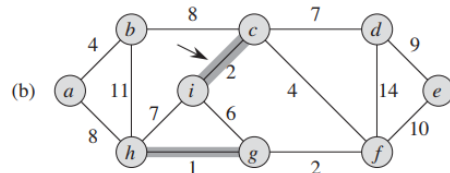
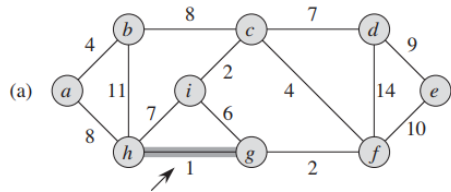
- یال‌های گراف را بر اساس وزن آنها بصورت صعودی مرتب می‌کنیم
- یال‌ها با وزن کمتر را به MST اضافه می‌کنیم به شرط اینکه حلقه ایجاد نشود
- هرگاه $v-1$ یال اضافه کردیم الگوریتم به پایان می‌رسد

MST-KRUSKAL(G, w)

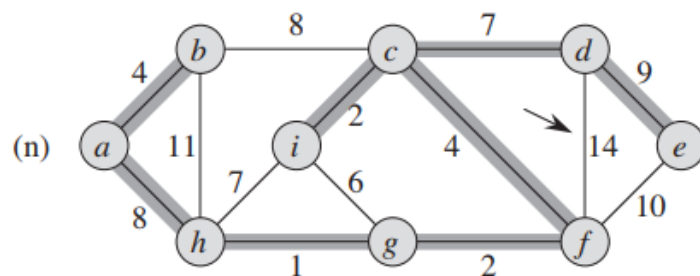
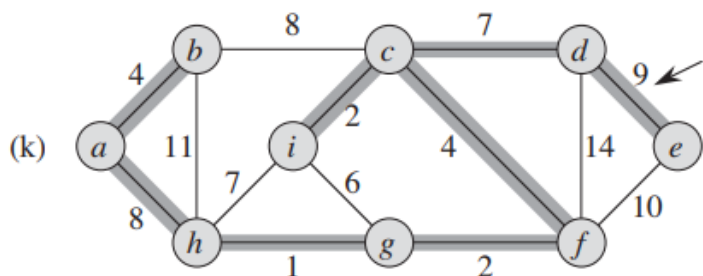
```
1   $A = \emptyset$ 
2  for each vertex  $v \in G.V$ 
3      MAKE-SET( $v$ )
4  sort the edges of  $G.E$  into nondecreasing order by weight  $w$ 
5  for each edge  $(u, v) \in G.E$ , taken in nondecreasing order by weight
6      if FIND-SET( $u$ )  $\neq$  FIND-SET( $v$ )
7           $A = A \cup \{(u, v)\}$ 
8          UNION( $u, v$ )
9  return  $A$ 
```

الگوریتم Kruskal برای ایجاد MST

- یال‌های گراف را بر اساس وزن بصورت صعودی مرتب می‌کنیم
- یال‌ها با وزن کمتر را به MST اضافه می‌کنیم به شرط اینکه حلقه ایجاد نشود
- هرگاه ۷-۱ یال اضافه کردیم الگوریتم به پایان می‌رسد



الگوریتم Kruskal برای ایجاد MST



MST-KRUSKAL(G, w)

- 1 $A = \emptyset$
- 2 **for** each vertex $v \in G.V$
- 3 MAKE-SET(v)
- 4 sort the edges of $G.E$ into nondecreasing order by weight w
- 5 **for** each edge $(u, v) \in G.E$, taken in nondecreasing order by weight
- 6 **if** FIND-SET(u) $\neq$ FIND-SET(v)
- 7 $A = A \cup \{(u, v)\}$
- 8 UNION(u, v)
- 9 **return** A

الگوریتم Kruskal برای ایجاد MST

MST-KRUSKAL(G, w)

```

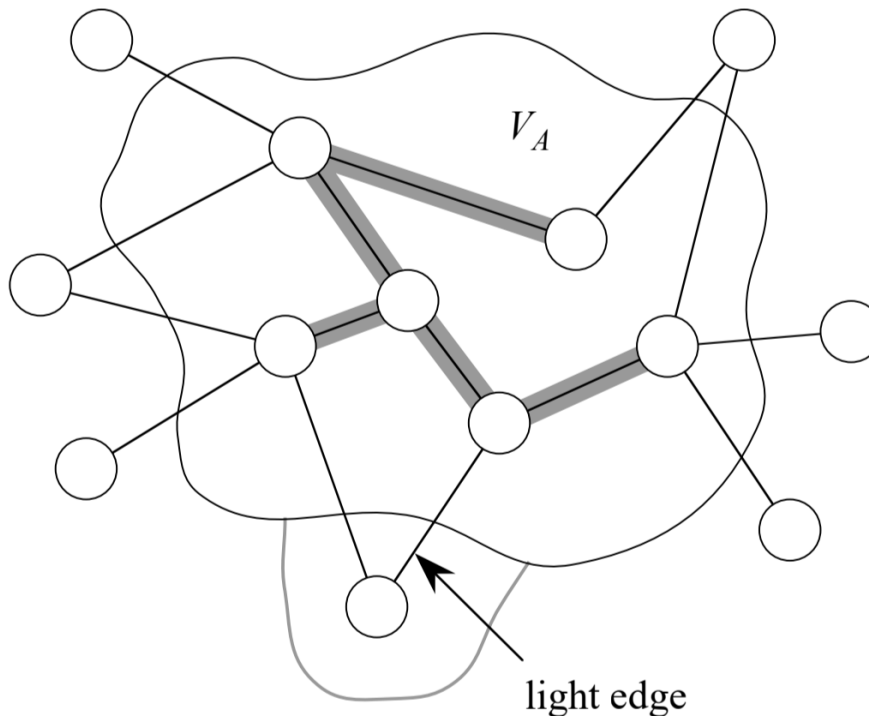
1   $A = \emptyset$ 
2  for each vertex  $v \in G.V$ 
3      MAKE-SET( $v$ )
4  sort the edges of  $G.E$  into nondecreasing order by weight  $w$ 
5  for each edge  $(u, v) \in G.E$ , taken in nondecreasing order by weight
6      if FIND-SET( $u$ )  $\neq$  FIND-SET( $v$ )
7           $A = A \cup \{(u, v)\}$ 
8          UNION( $u, v$ )
9  return  $A$ 

```

The running time of Kruskal's algorithm for a graph $G = (V, E)$ depends on how we implement the disjoint-set data structure. We assume that we use the disjoint-set-forest implementation of Section 21.3 with the union-by-rank and path-compression heuristics, since it is the asymptotically fastest implementation known. Initializing the set A in line 1 takes $O(1)$ time, and the time to sort the edges in line 4 is $O(E \lg E)$. (We will account for the cost of the $|V|$ MAKE-SET operations in the **for** loop of lines 2–3 in a moment.) The **for** loop of lines 5–8 performs $O(E)$ FIND-SET and UNION operations on the disjoint-set forest. Along with the $|V|$ MAKE-SET operations, these take a total of $O((V + E) \alpha(V))$ time, where α is the very slowly growing function defined in Section 21.4. Because we assume that G is connected, we have $|E| \geq |V| - 1$, and so the disjoint-set operations take $O(E \alpha(V))$ time. Moreover, since $\alpha(|V|) = O(\lg V) = O(\lg E)$, the total running time of Kruskal's algorithm is $O(E \lg E)$. Observing that $|E| < |V|^2$, we have $\lg |E| = O(\lg V)$, and so we can restate the running time of Kruskal's algorithm as $O(E \lg V)$.

الگوریتم Prim برای ایجاد MST

- Builds one tree, so A is always a tree.
- Starts from an arbitrary “root” r .
- At each step, find a light edge crossing cut $(V_A, V - V_A)$, where $V_A =$ vertices that A is incident on. Add this edge to A .



[Edges of A are shaded.]

الگوریتم Prim برای ایجاد MST

How to find the light edge quickly?

Use a priority queue Q :

- Each object is a vertex in $V - V_A$.
- Key of v is minimum weight of any edge (u, v) , where $u \in V_A$.
- Then the vertex returned by EXTRACT-MIN is v such that there exists $u \in V_A$ and (u, v) is light edge crossing $(V_A, V - V_A)$.
- Key of v is ∞ if v is not adjacent to any vertices in V_A .

The edges of A will form a rooted tree with root r :

- r is given as an input to the algorithm, but it can be any vertex.
- Each vertex knows its parent in the tree by the attribute $v.\pi = \text{parent of } v$.
 $v.\pi = \text{NIL}$ if $v = r$ or v has no parent.
- As algorithm progresses, $A = \{(v, v.\pi) : v \in V - \{r\} - Q\}$.
- At termination, $V_A = V \Rightarrow Q = \emptyset$, so MST is $A = \{(v, v.\pi) : v \in V - \{r\}\}$.

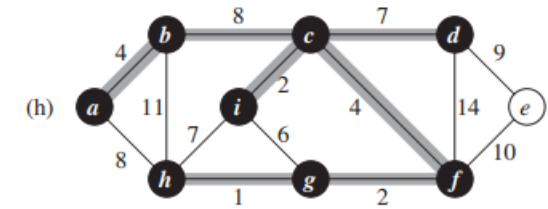
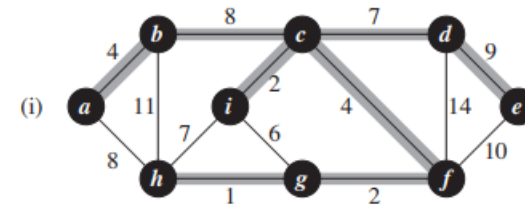
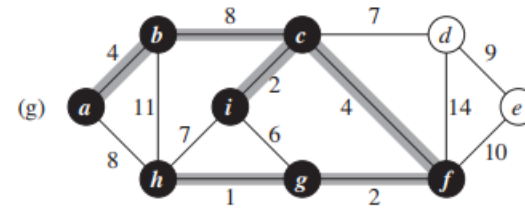
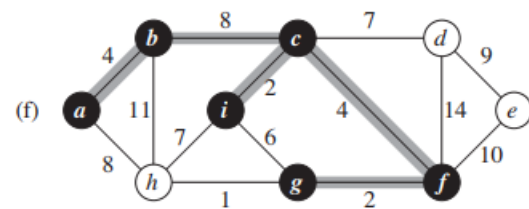
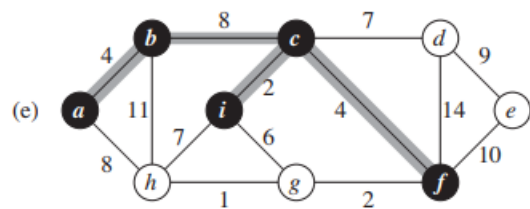
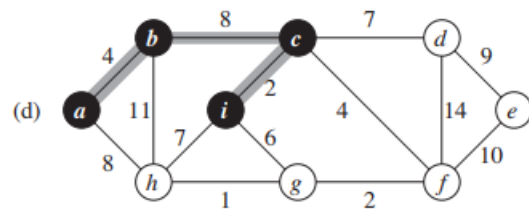
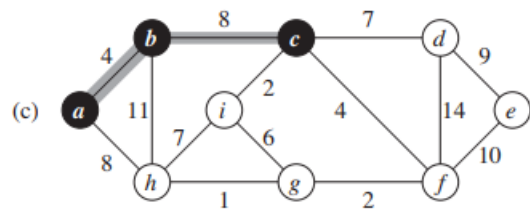
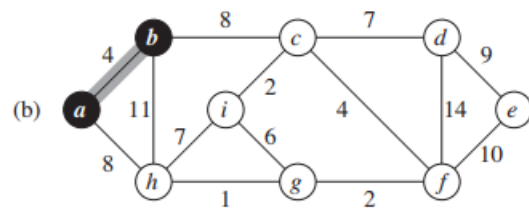
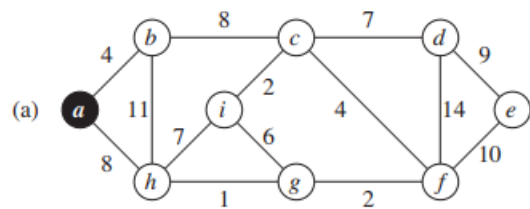
الگوریتم Prim برای ایجاد MST

1. از بین گره‌های گراف یک گره v را به دلخواه انتخاب کرده قرار می‌دهیم $S = \{v\}$

2. از بین یال‌های گذرکننده از $(S, V - S)$ یال با وزن کمینه را انتخاب و به MST اضافه می‌کنیم

3. راس طرف $V-S$ را به S اضافه می‌کنیم

4. تا زمانی که $S=V$ نشده است به گام ۲ برو



الگوریتم Prim برای ایجاد MST

1. از بین گره‌های گراف یک گره v را به دلخواه انتخاب کرده قرار می‌دهیم $S = \{v\}$

2. از بین یال‌های گذرکننده از $(S, V - S)$ یال با وزن کمینه را انتخاب و به MST اضافه می‌کنیم

PRIM(G, w, r)

$Q = \emptyset$

for each $u \in G.V$

$u.key = \infty$

$u.\pi = \text{NIL}$

INSERT(Q, u)

DECREASE-KEY($Q, r, 0$) // $r.key = 0$

while $Q \neq \emptyset$

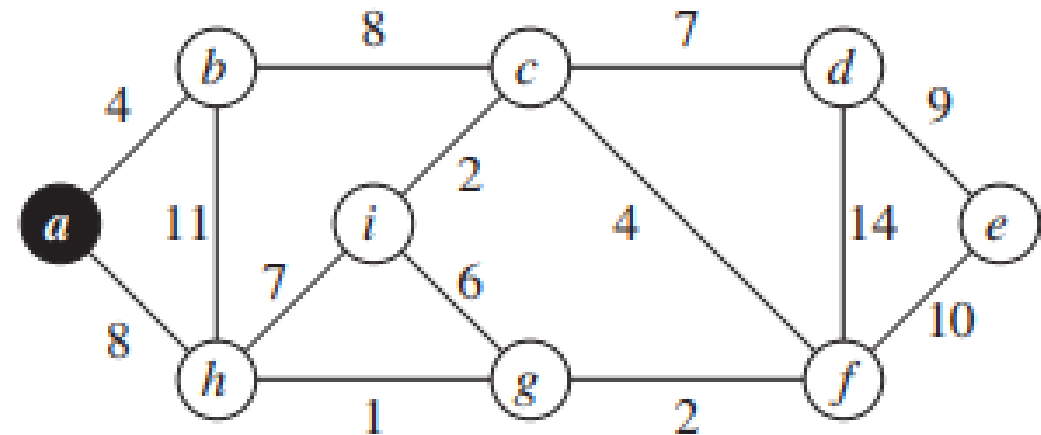
$u = \text{EXTRACT-MIN}(Q)$

for each $v \in G.Adj[u]$

if $v \in Q$ and $w(u, v) < v.key$

$v.\pi = u$

DECREASE-KEY($Q, v, w(u, v)$)



تحلیل زمان اجرای Prim با binary min heap

PRIM(G, w, r)

$Q = \emptyset$

for each $u \in G.V$

$u.key = \infty$

$u.\pi = \text{NIL}$

INSERT(Q, u)

DECREASE-KEY($Q, r, 0$) // $r.key = 0$

while $Q \neq \emptyset$

$u = \text{EXTRACT-MIN}(Q)$

for each $v \in G.Adj[u]$

if $v \in Q$ and $w(u, v) < v.key$

$v.\pi = u$

DECREASE-KEY($Q, v, w(u, v)$)

binary min-heap

BUILD-MIN-HEAP $O(V)$

$|V|$ times
EXTRACT-MIN $O(\lg V)$ $O(V \lg V)$
 $O(E)$

DECREASE-KEY $O(\lg V)$ $O(E \lg V)$

$$O(V \lg V + E \lg V) = O(E \lg V)$$

تحلیل زمان اجرای Prim با binary min heap

- Suppose we could do DECREASE-KEY in $O(1)$ amortized time.

Then $\leq |E|$ DECREASE-KEY calls take $O(E)$ time altogether $\Rightarrow$ total time becomes $O(V \lg V + E)$.

In fact, there is a way to do DECREASE-KEY in $O(1)$ amortized time: Fibonacci heaps, in Chapter 19.